

Installation of the FLAIR HUB Algorithm

Author: Constantin STOIAN

This document presents the installation of the FLAIR HUB algorithm [Garioud et al., 2025¹] proposed by IGN (French National Mapping agency) on a Windows computer.

For more information about the description of the FLAIR HUB algorithm, please follow the link:

<https://github.com/IGNF/FLAIR-HUB>

For more information about the *'models'*, which allow inference on a new input (e.g., an orthophoto), please refer to the link:

<https://huggingface.co/collections/IGNF/flair-models>

The following steps must be followed:

1. Install the "Miniconda3" software and accept Anaconda's "Terms of Service" (ToS)!
2. Create your working environment dedicated to FLAIR-HUB:
 - Open "Anaconda Prompt" (base) C:\Users\yourNAME> and run the command: *'conda create -n flairhub python=3.11'*; (The FLAIR-HUB repository on GitHub requires Python ≥ 3.11 , but if your flair hub environment uses a lower version (for example Python 3.10.18), then you must reinstall Git to update it to Python 3.11.)
 - Activate the environment with the command: *'conda activate flairhub'*:
 - The resulting prompt should be: (flairhub) C:\Users\yourNAME>
3. Install git via conda

The FLAIR-HUB repository is hosted on GitHub.

The command *'git clone https://github.com/IGNF/FLAIR-HUB.git'* allows to copy the entire project onto your PC, so you can modify and use it. Git works like a "GPS" for the code: it lets you retrieve exactly the version of the project you need.

The command to install Git is: *'conda install git -y'*. This downloads approximately 117.7 MB of packages.

4. Install PyTorch

Depending on the computer available, the algorithm can be installed with either the CPU inference option or the GPU inference option.

The segmentation of an orthophoto (inference) mainly consists of performing matrix multiplications, convolutions, summations, and repeated transformations on every pixel. Modern CPUs are very fast: they have 4 to 12 cores (sometimes 16–32 on workstations) designed for diverse tasks (logic,

¹ Anatol Garioud, Sébastien Giordano, Nicolas David, Nicolas Gonthier, 2025. FLAIR-HUB: Large-scale Multimodal Dataset for Land Cover and Crop Mapping, <https://doi.org/10.48550/arXiv.2506.07080>

sequential processing, multitasking). They are very efficient on one complex task, but not optimized for performing millions of identical operations simultaneously.

In contrast, a GPU (especially NVIDIA) contains thousands of specialized cores (CUDA cores), capable of executing the same operation on thousands of data points in parallel. As an example, for an orthophoto of several gigabytes in RAM, the inference time on a CPU ranges from 6 to 9 hours, whereas inference on a GPU takes a few minutes!

In this document, we explain how to install PyTorch for CPU-only usage. (GPU installation was not tested due to the lack of suitable hardware.)

WARNING! The algorithm requires 16 GB of dedicated RAM continuously to run properly! A configuration with at least 32 GB of RAM is recommended.

'*pip*' is the Python package manager. It is used to install Python libraries from PyPI (Python Package Index) or from other sources such as GitHub, as is the case here. FLAIR-HUB is not directly available through '*conda*'.

Therefore, the command to install PyTorch is: '*pip install torch torchvision torchaudio*'

5. Install FLAIR-HUB from GitHub

The command is: '*pip install git+https://github.com/IGNF/FLAIR-HUB.git*'

This clones FLAIR-HUB from the IGN GitHub repository onto your computer. This allows Python to download the code from GitHub and install it as a package.

Summary of the preparatory workflow

- **Git** → Download the FLAIR-HUB source code from GitHub.
- **pip** → Install this code into your Python environment so you can use it directly.
- **Conda** → Create an isolated environment (here *flairhub*) with the required Python version and packages, ensuring everything works without conflicts with other projects.

6. Download the "models" files

- Create the "models" folder in: C:/Users/yourNAME/FLAIR-HUB. Inside, you need to download the '**model**' that the algorithm will use.
- For orthophotos, this one can be used: https://huggingface.co/IGNF/FLAIR-HUB_LC-A_RGB_swinbase-upernet.safetensors

7. Create a *.yaml* configuration file

This file serves as the command file used to run inference with the trained model.

The project provides a file located at: ...\\FLAIR-HUB\\configs\\config_zonal_detection.yaml

which corresponds to the "Land Cover" model, but this file must be edited and adapted according to our needs.

The parameters required to perform inference on orthophotos can be found in the model repository: https://huggingface.co/IGNF/FLAIR-HUB_LC-A_RGB_swinbase-upernet

Edit the file (config_zonal_detection.yaml):

7.1. The first block:

If the option '**cog_conversion: True**' is enabled, FLAIR-HUB will convert the entire prediction **into COG (Cloud Optimized GeoTIFF)** format. This operation implies a complete rewrite of the final TIFF and requires double read-write operations to reorganize the file into blocks, therefore using a large amount of RAM. For large files, it is better to set: '**cog_conversion: False**' and then convert to 'COGeoTIFF afterward using *gdal*.

```
# =====  
# I/O  
# =====  
# Folder where the output raster will be saved  
  
output_path: "C:/Users/.../inference/"  
  
# Name of the raster output file  
output_name: "..."  
  
# Output file  
write_dataframe: False  
output_type: argmax # [argmax / class_prob]  
cog_conversion: False/True
```

7.2. The second block: "Model & Inference"

This section specifies the model that will be used for inference and that has previously been downloaded and installed in the directory: **.../FLAIR-HUB/models/**.

In our case, we use the model '**LC-A_RGB**', which is pre-trained for orthophotos (20 cm resolution), since we only have the R, G, and B channels.

If you are on **Windows**, the parameter *num_worker*: must be set to **0**.

On **Linux**, it should typically be between **4 and 12**.

num_worker corresponds to the **number of parallel processes** used to:

- load the images,
- split them into tiles,
- apply transformations.

The more workers you allocate, the faster the data is prepared and the faster inference runs. However, Windows handles multiprocessing differently from Linux: each worker must recreate the entire Python environment and reload all libraries (torch / GDAL / rasterio, etc.), which can cause crashes.

Therefore, on Windows, it is recommended to set: *num_worker*: **0**

The model cannot process the entire image in one go, RAM and VRAM would be saturated, so it processes the data tile by tile.

The tile size sent into the segmentation model is 2048 pixel, meaning the image is split into 2048 ×

2048-pixel tiles.

The parameter margin: 512 indicates that tiles **overlap** by 512 pixels to avoid edge artifacts.

The output resolution for which the model **‘FLAIR-HUB_LC-A_RGB’** was trained is **0.2 m/pixel**. To obtain accurate results, this value must be preserved.

- Increasing the resolution will push the algorithm to detect excessive detail, potentially adding unwanted classes (e.g., a tiny patch of vegetation on a rooftop may produce a “vegetation hole” in the building class).
- Decreasing the resolution will reduce precision and cause class mixing.

```
# =====  
# Model & Inference  
# =====  
# Path to the pre-trained model  
  
model_weights: "C:/Users/.../FLAIR-HUB/models/FLAIR-HUB_LC-A_RGB_swinbase-upernet.safetensors"  
  
# Runtime Settings  
use_gpu: True/False  
num_worker: 0/10  
  
# Labels dataset to use (no labels, we use the pre-trained model)  
  
# Resolution and patches size  
img_pixels_detection: 2048  
margin: 512  
output_px_meters: 0.2
```

7.3. The third block: “Model Framework”

For inference, only the *monotemp_model* is used, as it processes a single image.

This section defines the network architecture into which the *.safetensors* weights will be loaded:

- the choice of the Swin-B backbone,
- the UPerNet architecture used for segmentation,
- the internal network parameters matching the model weights (here LC-A),
- and the indication that the model does not combine multiple dates.

```
# =====  
# Model Framework  
# =====  
models:  
  monotemp_model:  
    name: timm-swin_base_patch4_window12_384  
    arch: swin_base_patch4_window12_384-upernet  
    encoder_widths: [128, 256, 512, 1024]  
    decoder_channels: [512, 256, 128, 64]  
    num_classes: 20  
    backbone_pretrained: True  
    use_temporal_encoder: False
```

The *multitemp_model* block corresponds to multi-temporal models, which analyze several dates simultaneously, such as LC-ASM, LC-AD, LC-MT.

```

multitemp_model:
  name: timm-swin_base_patch4_window12_384
  encoder_widths: [128, 256, 512, 1024]
  decoder_channels: [512, 256, 128, 64]
  num_classes: 20
  backbone_pretrained: True
  use_temporal_encoder: False
  temporal_agg: none
  output_stride: 32

```

7.4. The fourth block: “Modalities (inputs used for inference)”

In this block, you activate the modality that will be used for inference. For our use case, this is **AERIAL_RGBI**. The input channels used for orthophotos are (1, 2, 3). Then, you must specify the full path to your orthophoto.

It is very important to correctly set the parameters ‘*means*’ and ‘*stds*’, which differ depending on the model.

These represent the normalization statistics used during training:

- means = per-channel pixel value averages
- stds = per-channel standard deviations

Deep learning normalization formula:

pixel_normalized = (pixel – mean) / std

It is recommended to keep the values associated with the model you are using (here: **LC-A_RGB_swinbase-upernet**).

```

# =====
# Modalities (inputs used for inference)
# =====
modalities:
  inputs:
    AERIAL_RGBI      : True
    AERIAL-RLT_PAN   : False
    DEM_ELEV         : False
    SPOT_RGBI        : False
    SENTINEL2_TS     : False
    SENTINEL1-ASC_TS : False
    SENTINEL1-DESC_TS : False

normalization:
  norm_type: custom

inputs_channels:
  AERIAL_RGBI: [1, 2, 3]
  AERIAL-RLT_PAN: [1]
  DEM_ELEV: [4, 1, 2]
  SPOT_RGBI: [4, 1, 2]
  SENTINEL2_TS: [1,2,3,4,5,6,7,8,9,10]
  SENTINEL1-ASC_TS: [1,2]
  SENTINEL1-DESC_TS: [1,2]

```

```
# === YOUR MAIN INPUT ===
AERIAL_RGBI:
  input_img_path: c:/Users.../image_orthophoto.tif
  channels: [1, 2, 3]
  normalization:
    type: custom
    means: [105.66, 111.35, 102.18]
    stds: [52.23, 45.62, 44.30]
  reference_modality: True
```

7.5. The fifth block: “Labels”

Create the labels block and add the class **AERIAL_LABEL-COSIA**.
This must match exactly the task name defined in the tasks block.

This section defines the semantic dictionary that links:

- the numerical indices predicted by the model, and
- the real geographic meaning of the classes (building, water, etc.).

COSIA stands for “*Classification de l’Occupation du Sol par Intelligence Artificielle*” (IGN), an official land-cover nomenclature.

```
# =====
# Labels
# =====
labels:
  - AERIAL_LABEL-COSIA # In the COSIA model, there are 19 labels (therefore 19 LC classes),
                        # but as indicated on the repository
                        # https://huggingface.co/IGNF/FLAIR-HUB_LC-A_RGB_swinbase-upernet
                        # only the first 15 classes (0 to 14) are detected.
```

```
labels_configs:
AERIAL_LABEL-COSIA:
  num_classes: 19
  value_name:
    0: building
    1: greenhouse
    2: swimming_pool
    3: impervious surface
    4: pervious surface
    5: bare soil
    6: water
    7: snow
    8: herbaceous vegetation
    9: agricultural land
    10: plowed land
    11: vineyard
    12: deciduous
    13: coniferous
    14: brushwood
    15: clear cut
    16: ligneous
    17: mixed
    18: undefined
```

7.6. The sixth block: “Tasks”

Create the ‘**tasks**’ block, set ‘**name**’: AERIAL_LABEL-COSIA, and activate the class with ‘**active: True**’.

A **task** is a **prediction task** performed by the model during inference. Depending on the model used, it may predict several outputs in parallel (land cover, buildings, roads, vegetation, water, etc.).

- If ‘**active: True**’, the task is **enabled**, and the model will compute that output.
- If ‘**active: False**’, the task is **disabled**, and it will neither be computed nor appear in the results.

```
# =====  
# Tasks  
# =====  
tasks:  
- name: AERIAL_LABEL-COSIA      # Labelling task for semantic segmentation (land cover classes)  
  active: True  
    0: building  
    1: greenhouse  
    2: swimming_pool  
    3: impervious surface  
    4: pervious surface  
    5: bare soil  
    6: water  
    7: snow  
    8: herbaceous vegetation  
    9: agricultural land  
    10: plowed land  
    11: vineyard  
    12: deciduous  
    13: coniferous  
    14: brushwood  
    15: clear cut  
    16: ligneous  
    17: mixed  
    18: undefined
```

7.7. The seventh block: “Saving”

If you choose ‘**save_predictions: True**’, you may encounter issues when working with very large orthophotos. FLAIR-HUB generates one TIFF prediction per tile, and for multi-gigabyte files it creates very large temporary prediction files, which can cause the algorithm to crash if there is not enough available RAM. During inference, there is no need to save a model, and the same applies to logs: they are small but unnecessary if you are only performing inference.

```
# =====  
# Saving  
# =====  
saving:  
  cp_csv_and_conf_to_output: True  
  save_predictions: False/True  
  save_model: False  
  save_training_logs: False
```

7.8. The eighth block: “Hyperparams”

The parameters used here are those provided on the model’s official page:

https://huggingface.co/IGNF/FLAIR-HUB_LC-A_RGB_swinbase-upernet

```
# =====  
# Hyperparams  
# =====  
hyperparams:  
  batch_size: 5  
  learning_rate: 5e-5  
  weight_decay: 0.0  
  num_epochs: 150  
  optimizer: AdamW (betas=[0.9, 0.999], weight_decay=0.01)  
  seed: 2025
```

7.9. The ninth block: “Rasterio profile”

This section is very useful because it overrides the default settings from:

src/flair_zonal_detection/inference.py.

When dealing with large input images, it prevents inference from stopping and enables the use of BIGTIFF writing.

```
# =====  
# Rasterio profile  
# =====  
rasterio_profile:  
  driver: GTiff  
  dtype: uint8  
  count: 1  
  tiled: True  
  blockxsize: 256  
  blockysize: 256  
  compress: lzw  
  BIGTIFF: YES
```

To launch the inference, the exact command to type in the conda environment is:

```
(flairhub) C:\Users\yourNAME> python src\flair_hub\main.py --config  
configs\config_zonal_detection.yaml
```